Software Engineering : Project 1

Apache Roller – Architectural Analysis and Refactoring

Introduction

This project focuses on understanding, analyzing, and improving the design of **Apache Roller**, a large-scale, open-source, Java-based blogging platform. Apache Roller is a mature enterprise system that supports multi-user blogging, role-based access control, and content search through indexing. Due to its size and complexity, the project emphasizes **design recovery, architectural reasoning, and controlled refactoring**, rather than feature development.

The work is structured into multiple tasks, each addressing a specific aspect of software design and maintenance. To improve clarity and navigability, this document provides a **high-level summary of all tasks**, while detailed explanations, diagrams, analyses, and results are maintained in **separate task-wise documents**.

Task 1: Architectural Mapping and Design Recovery

Objective

The goal of Task 1 is to recover and document the architectural structure of Apache Roller. Since the system is large and complex, we decompose it into **three major subsystems** to make analysis manageable and meaningful.

Subsystems Analyzed

1. Weblog and Content Subsystem
2. User and Role Management Subsystem
3. Search and Indexing Subsystem

Each subsystem was analyzed **independently**, with attention given to how they interact with one another.

Work Performed

- Identification of key Java classes and interfaces for each subsystem
- Documentation of responsibilities and interactions within each subsystem

- Creation of UML diagrams using PlantUML / StarUML to model class relationships
- Analysis of design strengths and weaknesses
- Explicit documentation of assumptions made during modeling

To understand the impact of automation, a small, clearly scoped part of one subsystem was analyzed both **manually** and with **LLM assistance**, followed by a comparison of effort, correctness, and completeness.

Detailed Documentation

Refer to the dedicated Task 1 document for subsystem descriptions, class-level explanations, UML diagrams, and comparative analysis between manual and LLM-assisted design recovery.

Subsystems:

- User Role Management (documentation)
- Weblog and Content
- Indexing and Searching

Manual Vs LLM (Documentation)

Task 2: Design Smells and Code Metrics Analysis

Task 2A: Design Smells Analysis (Documentation)

Objective

The objective of Task 2A is to identify **design-level smells** that indicate deeper architectural issues, rather than focusing only on fine-grained code smells.

Work Performed

- Static analysis tools such as SonarQube and DesigniteJava were used to detect potential design smells
- UML diagrams from Task 1 were used to reason about architectural problems
- Automated tool outputs were interpreted manually to avoid false positives
- A curated list of significant design smells was created, supported by evidence

The identified smells formed the foundation for all subsequent refactoring work.

Task 2B: Code Metrics Analysis (Documentation)

Objective

The goal of Task 2B is to quantitatively evaluate the quality of the Apache Roller codebase using software metrics and to understand how these metrics can guide refactoring decisions.

Work Performed

- Code metrics were collected using static analysis tools
- Up to six metrics were selected, including object-oriented metrics such as those proposed by Chidamber and Kemerer
- Each metric was interpreted in terms of maintainability, design quality, and potential performance implications
- The results were used to justify the need for refactoring

Detailed Documentation

Separate documents are provided for Task 2A (design smells) and Task 2B (code metrics), including tool outputs, explanations, and interpretations.

Task 3: Refactoring

Task 3A: Manual Refactoring

Objective

The objective of Task 3A is to address the design smells identified in Task 2A through **manual, class-level refactoring**, without changing the observable behavior of the system.

Work Performed

- Significant refactorings were applied at the class and subsystem level
- Refactorings focused on improving encapsulation, modularity, and separation of concerns
- Existing test cases were executed after refactoring to ensure behavioral correctness
- Core functionality was manually verified where required

Only manually implemented changes were committed to the main codebase, as required by the project guidelines.

Subsystem	Design Smell Refactored	Primary Files Involved	Refactoring Action & Improvement
Weblog & Content	God Class / High WMC	Weblog.java, WeblogEntry.java, JPAWeblogEntryManagerImpl	**Impact:** Complexity (WMC) was reduced by ~15%. Business logic was extracted from POJOs into service managers, enforcing the Single Responsibility Principle.
Weblog & Content	Complex Method (High CC)	HTMLSanitizer.java, PageServlet.java	**Impact:** Cyclomatic Complexity was reduced from CC 34 to CC 1. Deeply nested conditionals were replaced with helper methods and delegation.
Search & Indexing	Magic Numbers	FieldConstants.java, SearchOperation.java, LuceneIndexManagerImpl	**Impact:** Centralized over 30 hardcoded Lucene field strings into `FieldConstants`. This eliminated string-typing bugs and enabled global field updates.
Search & Indexing	Broken Hierarchy	IndexOperation.java, AddEntryOperation.java, SearchOperation.java	**Impact:** Introduced a standardized 3-level hierarchy (Base → Specialization → Concrete). Applied the Template Method Pattern to ensure consistent Lucene task execution.

User & Role	Insufficient Modularization	UserManager.java, UserValidator.java, UserPasswordService.java	**Impact:** Extracted logic from `JPAUserManagerImpl` into dedicated service classes, reducing class bloat and improving testability.
User & Role	Deficient Encapsulation	User.java, UserRole.java, WeblogPermission.java	**Impact:** Replaced direct field access with a centralized Permission Model, reducing cyclic dependencies between Users and Roles.
Search & Indexing	Unutilized Abstraction	IndexManager.java, SearchOperation.java	**Impact:** Simplified interface–implementation relationships by removing redundant legacy classes and consolidating the indexing API to reduce cruft.

Task 3B: Post-Refactoring Metrics (Documentation)

Objective

This task evaluates the impact of manual refactoring by re-running the code metrics analysis and comparing results with the pre-refactoring state.

Work Performed

- Metrics from Task 2B were recomputed after refactoring
- Improvements and regressions were analyzed
- Trade-offs between different metrics were discussed, acknowledging that improving one metric can worsen another

---

Task 3C: Automating the Refactoring Pipeline using LLMs (Documentation)

Objective

The goal of Task 3C is to design and document an **automated refactoring pipeline** that uses LLMs to detect design smells, generate refactoring suggestions, and create pull requests.

Work Performed

- A pipeline was designed that integrates smell detection, LLM-based refactoring, and PR generation
- The pipeline workflow and decision logic were documented
- A flowchart was created to explain how the pipeline handles large files and context
- Suggested refactorings were documented but **not merged** into the main codebase

This task focuses on automation design rather than correctness of generated code.

Task 4: Agentic Refactoring on a Single File (Documentation)

Objective

The objective of Task 4 is to explore **agentic refactoring**, where an automated system performs smell identification, refactoring planning, and refactoring execution in a structured manner.

Work Performed

- An existing agentic refactoring tool was used
- A small set of Java source files was selected
- The agentic workflow was documented, including smell detection, planning, and execution stages
- Original and refactored code snippets were compared

The suggested changes were documented only and were not applied to the main repository.

Task 5: Comparative Refactoring Analysis (Manual vs LLM vs Agentic) (Documentation)

Objective

Task 5 provides a unified qualitative comparison of **manual refactoring**, **LLM-assisted refactoring**, and **agentic refactoring**.

Work Performed

- Two design smells were selected from the earlier analysis
- For each smell, the same representative code was analyzed across all three approaches
- Refactoring outcomes were compared across multiple dimensions, including clarity, conciseness, design quality, faithfulness, architectural impact, and human versus automation judgment
- Strengths and limitations of each approach were discussed

This task emphasizes **engineering judgment** and reflective analysis rather than implementation.

Repository Organization and Navigation

To improve readability and evaluation, the documentation is organized as follows:

- This document provides a **high-level overview of all tasks**
- Each task has a **separate, detailed document** containing in-depth explanations, diagrams, tables, and screenshots
- UML diagrams, tool outputs, and supplementary artifacts are stored alongside their respective task documents
- The main codebase contains **only manual refactoring changes**, in accordance with project rules

---

Team Members

- **Sanapathi Kishore Naidu** (2025204002)
- **Vadali S S Bharadwaj** (2025204012)
- **Kondeboina Anil Kumar** (2025204018)
- **Deepti Koranga** (2025204033)
- **Rahul Ebenezer Chand** (2025204037)

Team Contribution

Task1

- User Role Management - (Rahul Chand)
- Weblog and Content - (Bharadwaj)
- Indexing and Searching - (Kishore Naidu)
- Manual Vs LLM - (Deepti)

Task 2

- **Task_2A** - (Anil and Deepti)
- **Task_2B** - (Anil)

Task 3

- **Task_3A**

  - User Role Management - (Rahul and Deepti)

  - Weblog and Content - (Bharadwaj)

  - Indexing and Searching - (Anil and Kishore)

- **Task_3B** - (Anil and Bharadwaj)
- **Task_3C** - (Bharadwaj)

Task 4 - (Rahul)

Task 5 - (Kishore)

Final Documentation - (Deepti and Kishore)

Conclusion

This structured approach to documentation was chosen to manage the complexity of Apache Roller and to clearly separate architectural analysis, empirical evaluation, and refactoring activities. By combining manual reasoning with automated and agentic approaches, the project highlights both the strengths and limitations of modern software engineering tools when applied to real-world systems.
